

Git und Gitlab

Programmierprojekt, Timo Stadler

Was sind Git und GitLab?

- **Git** ist ein verteiltes Versionskontrollsystem, das Quellcode-Änderungen lokal auf dem Rechner nachverfolgt.
- Think of **Git** as the engine of a car, and **GitLab** as the garage where you park and service that car. You can use the engine on its own, but the garage gives you a secure space, tools, and a team to work on it.

Was sind Git und GitLab?

The name "git" was given by Linus Torvalds when he wrote the very first version. He described the tool as "the stupid content tracker" and the name as (depending on your mood):

- random three-letter combination that is pronounceable, and not actually used by any common UNIX command. The fact that it is a mispronunciation of "get" may or may not be relevant.
- stupid. contemptible and despicable. simple. Take your pick from the dictionary of slang.
- "global information tracker": you're in a good mood, and it actually works for you. Angels sing, and a light suddenly fills the room.
- "goddamn idiotic truckload of sh*t": when it breaks

Wofür?

- **Problem:**

- Manuelle Dateiversionierung („Projekt_v2_final_FINAL.zip“) ist fehleranfällig.
- Wer hat wann welche Änderungen gemacht?

- **Lösung:**

- Verteiltes Versionskontrollsystem
- Jeder Entwickler hat die komplette Historie

- **Vorteile:**

- Vollständige Historie, paralleles Arbeiten (Branching), lückenlose Nachvollziehbarkeit.

Git Grundkonzepte und Begriffe

- **Working Directory:** Lokaler Stand des Projekts auf dem eigenen PC. Wie gewohnt bearbeiten von Dateien im Editor.
- **Staging Area:** Gezieltes Auswählen welche Änderungen einem **Commit** hinzugefügt werden sollen
 - `git add .` → Hinzufügen von allen Änderungen zum commit
 - `git add src/main.c` → Nur angegebene Datei hinzufügen
- **Repository:**
 - Jeder Commit wird über einen eindeutigen Hash indentifiziert.
 - `git commit -m "Nachricht"` → speichert den Schnappschuss dauerhaft in der lokalen Datenbank
 - `git pull` → Download der Änderungen aus dem Remote-Repository
 - Immer einen "pull" vor einem "push" machen!
 - `git push` → Upload der lokalen Änderungen in das Remote-Repository

Grundkonzepte (Branching + Merging)

- **Branches:** Isolierte Entwicklungsumgebungen für Features oder Bugfixes.
 - In Git ist ein Branch lediglich ein beweglicher Zeiger auf einen bestimmten Commit. Das Erstellen oder Wechseln von Branches dauert Millisekunden.
 - `git branch feature-MeinNeuesFeature` → erstellt einen neuen Branch
 - `git checkout feature-MeinNeuesFeature` → wechselt zum neuen Branch
 - `git branch` → Prüfen auf welchem branch ich mich befinde
- **Merge & Rebase:** Zusammenführen von Änderungen.
 - **Merge:** Historie beider Branches bleibt vollständig erhalten, und Git erstellt (falls nötig) einen neuen **Merge-Commit**
 - `git checkout main`
 - `git pull`
 - `git merge feature-MeinNeuesFeature`
 - `git push origin main`
 - **Rebase:** Verschieben der Basis des Feature-Branches auf den aktuellsten Stand des Ziel-Branches. Commits werden sozusagen „abgeschnitten“ und oben auf den neuesten Stand neu aufgesetzt.
 - `git checkout feature-MeinNeuesFeature`
 - `git rebase main`
- Generell ist es Philosophie/Geschmackssache, ob merge oder rebase verwendet wird.
 - → **Merge verwenden für Transparenz!**

Weitere Befehle

- **git status**: Zeigt den aktuellen Zustand des Arbeitsverzeichnisses
- **git log**: Zeigt die Commits chronologisch rückwärts an – inklusive Autor, Datum, Commit-Hash und der vollständigen Commit-Message
- **git diff**: Zeigt die zeilenweisen Änderungen an, die noch **nicht** für den nächsten Commit vorgemerkt (git add) wurden.
- **git restore <datei>**: Verwirft lokale Änderungen in einer Datei und setzt sie auf den Zustand des letzten Commits zurück.
- **git cherry-pick <a1b2c3d>**: Exakte Änderungen aus einem Commit herausfischen. Notwendig z.B. für Bugfixes oder wenn Commits auf falschen Branches gemacht wurden.
- **git init**: Neues lokales Repository erstellen

Woher bekomme ich Git?

- Windows:
 - <https://git-scm.com/install/windows>
 - Das Standard GUI-Tool ist nicht zu empfehlen
 - Die Installation kommt mit einer Bash in der gängige Linux-Befehle ausgeführt werden können
- macOS:
 - `git --version` ins Terminal eingeben oder installieren über Homebrew
- Linux:
 - Installation über den Paketmanager eurer Distro
- Nützliche Tools für die Verwendung:
 - VS Code Extensions:
 - Generelle Git Integration in eurer IDE/Editor
 - Gitlens, Git Graph, Git History
 - GitLab
 - Standalone Tools:
 - GitKraken
 - Git GUI

Übung

- <https://learngitbranching.js.org>
- https://gitlab.oth-regensburg.de/users/sign_in
- **Account erstellen wenn noch nicht passiert!**
- Weitere Ressourcen:
 - <https://git-scm.com/learn>
 - Git Buch zur Erklärung der Basics und fortgeschrittene Themen
 - <https://ohshitgit.com/>
 - Gängige Fehlerbehandlungen
 - <https://docs.gitlab.com/tutorials/>
 - Infos zur Verwendung von GitLab (Möglich